

AirStems

Report de funcionalidad

Lluís Estapé · Musicathon by Musixmatch Pro 2026 · 2026-06-15

Qué es. Un instrumento que convierte la *separación de stems* y la *letra sincronizada* en un **performance en directo**. Una webcam te sigue las manos y, con gestos, remezclas en tiempo real las pistas de una canción (voz / batería / bajo / otros), con los cambios **cuadrados al beat** y la **letra en karaoke**.

Estado (2026-06-15). Núcleo **verificado offline**: separación local (Demucs), motor de audio **estéreo**, detección de beats sobre el stem de batería (129.2 BPM / 363 beats en la canción de prueba) y la app con tracking de manos + HUD + letra. **APIs**: Musixmatch (letra sincronizada y rich sync palabra-por-palabra) **verificada**; LALAL.AI *upload* verificado (la separación requiere la licencia premium del hackathon); Cyanite (BPM/tono/mood) **integrada**, a la espera del token.

Contents

1	Cómo funciona: dos flujos	1
2	Controles	2
3	El diferenciador: toggles cuadrados al beat	2
4	Motor de audio (StemEngine)	2
5	Letra / karaoke (lyrics.py)	3
6	Origen de los stems (desacoplado) y APIs de partners	3
7	HUD (lo que se ve en pantalla)	3
8	Componentes (ficheros)	3
9	Stack técnico	4
10	Cómo ejecutar	4
11	Pendiente / próximos pasos	4

1 Cómo funciona: dos flujos

Flujo de AUDIO (qué suena):

```
Cancion
|
v
Separador (Demucs o LALAL.AI)
|
v
stems/<cancion>/<vocals,drums,bass,other>.wav
|
v
StemEngine -> carga stems + detecta beat grid (librosa)
```

```
| (callback sounddevice, 44.1 kHz, bloque 512)
v
mezcla (gain por stem) -> filtro paso-bajo -> reverb -> tremolo -> SALIDA
```

Flujo de CONTROL (qué haces tú):

```
Webcam -> MediaPipe (2 manos) -> landmarks
|
|-- mano derecha : dedos arriba/abajo -> gain ON/OFF por stem (histéresis)
|-- mano izquierda: altura de la muñeca -> filtro paso-bajo
|               abrir / cerrar la mano -> reverb
|-- beat-sync (tecla b) -> los cambios de gain saltan al siguiente beat
|
Letra (.lrc por línea o rich sync palabra-por-palabra) -> karaoke sincronizado
```

2 Controles

Primero **haz clic en la ventana “AirStems”** para que tenga el foco del teclado.

Acción	Gesto / tecla
Play / pausa	espacio
Beat-sync ON/OFF	b
Salir	q
Stems 1–4 ON/OFF	mano derecha: subir/bajar cada dedo (índice→meñique)
Drop total / mezcla completa	puño / mano abierta
Filtro paso-bajo	mano izquierda: altura de la muñeca (abajo oscuro, arriba abierto)
Reverb	mano izquierda: abrir / cerrar la mano (abierta full, puño 0)

Los dedos usan **histéresis** (UP_MARGIN/DOWN_MARGIN) para no parpadear en el umbral; el filtro y la reverb usan **suavizado EMA** (SMOOTH).

3 El diferenciador: toggles cuadrados al beat

Al cargar la canción, **librosa** detecta el BPM y el tiempo de cada beat. Con beat-sync **ON**, cuando mutes o activas un stem el cambio **no es instantáneo**: se guarda como *pendiente* y se aplica en el **siguiente beat** (el callback de audio comprueba si cae un beat dentro del bloque actual; el *wrap* del bucle cuenta como *downbeat*). Así cada *drop* y cada entrada **caen a tiempo** y suenan intencionados. Con beat-sync **OFF**, los cambios son inmediatos. Las transiciones de gain usan una **rampa por bloque**, así que no hay clics.

4 Motor de audio (StemEngine)

- Reproduce en bucle N stems (**estéreo**, 44.1 kHz) en un callback de **sounddevice** (bloque 512).
- **Mezcla** = suma de (stem × gain) con rampa de gain por bloque, × master; efectos **por canal**.
- **Cadena de efectos** (reutilizada de `synth.py` de *Aetheric Geometry*):
 - Filtro **IIR paso-bajo** de 1.er orden (corte 200–8000 Hz según la altura de la mano).
 - **Reverb Schroeder** (4 comb filters, feedback 0.82), *wet* suavizado.
 - **Trémolo** (LFO 0.5–10 Hz).
- Expone: `position_seconds` (sincroniza la letra), `bpm`, `beat_pulse` (HUD) y `quantize`.

5 Letra / karaoke (lyrics.py)

Parsea un `.lrc` (timestamps `[mm:ss.xx]`) y devuelve la **línea activa** según la posición de reproducción del motor. Además soporta el **rich sync** de Musixmatch (`.richsync.json`): **timing palabra por palabra**, así que el HUD resalta la parte ya cantada (`current_karaoke`). Si hay rich sync se prefiere; si no, cae al `.lrc`.

6 Origen de los stems (desacoplado) y APIs de partners

El motor solo carga WAVs de `stems/<cancion>/`: le da igual quién los generó. Por eso las fuentes son **intercambiables**.

Módulo	Partner	Rol
<code>separate_local.py</code>	— (Demucs)	separación local/offline, sin key (desarrollo y <i>fallback</i>)
<code>lalalai.py</code>	LALAL.AI	separación en la nube (v1 API), integración oficial
<code>musixmatch.py</code>	Musixmatch	descarga la letra sincronizada (<code>.lrc</code>)
<code>cyanite.py</code>	Cyanite	BPM / tono / mood (librosa hace la rejilla de beats local)

7 HUD (lo que se ve en pantalla)

- Barra por stem (verde = ON, gris = off).
- Barras de **filtro** y **reverb**.
- **Pulso** circular en cada beat + **BPM** + estado de beat-sync.
- **Línea de letra** sincronizada.
- Contador de **FPS**.

8 Componentes (ficheros)

Fichero	Rol
<code>airstems.py</code>	bucle principal: cámara, tracking, mapeo gestos→params, HUD, teclas
<code>stem_engine.py</code>	StemEngine : audio en tiempo real, mezcla, efectos, beat grid + beat-sync
<code>lyrics.py</code>	parser LRC + línea actual
<code>separate_local.py</code>	separación local con Demucs (carga audio vía soundfile)
<code>lalalai.py</code>	cliente LALAL.AI v1 (upload → split → check → download)
<code>musixmatch.py</code>	cliente de letra sincronizada
<code>cyanite.py</code>	cliente de análisis (BPM/tono/mood)
<code>smoke_test.py</code>	verificación headless (sin cámara ni altavoz)
<code>gestures.py, renderer.py, config.py</code>	reutilizados de <i>Aetheric Geometry</i>

9 Stack técnico

Python · OpenCV · MediaPipe · sounddevice · soundfile · NumPy · librosa · (Demucs / torch para la separación local) · APIs: **LALAL.AI**, **Musixmatch**, **Cyanite**.

10 Cómo ejecutar

Con el venv de *Aetheric* (mediapipe 0.10.9, que sí tiene `mp.solutions`):

```
& "...\\Gestural-Harmonic-Mapping\\venv\\Scripts\\python.exe" "...\\AirStems\\airstems.py"
```

Separar otra canción:

```
python separate_local.py "C:\\ruta\\cancion.wav"    # local (Demucs)
python lalalai.py      "C:\\ruta\\cancion.wav"    # LALAL.AI (con key)
```

11 Pendiente / próximos pasos

- **Cyanite:** en cuanto llegue el token, BPM/tono/mood oficiales (`cyanite.py` ya implementa el flujo `fileUploadRequest` → `libraryTrackCreate` → análisis V7).
- **LALAL.AI:** correr una separación real al activarse la licencia premium del hackathon (hoy: `upload` verificado; Demucs cubre desarrollo y demo).
- **Mío (post-examen):** afinar umbrales de gestos en vivo, elegir tema y grabar el vídeo.
- **Stretch:** export de la remezcla a WAV · *pinch* = solo de un stem · volumen master con la distancia entre manos.